Build Your Own GenAIOps Agent — Complete Roadmap

Goal

Build a personal enterprise-style GenAIOps assistant that can:

- provision infrastructure
- manage prompts
- version agents
- run evaluations
- automate testing
- monitor AI systems
- analyze traces
- troubleshoot deployment issues
- explain Azure GenAIOps concepts
- help solve errors step-by-step

This project will evolve gradually through multiple stages.

Final Vision

Your final system should eventually work like this:

```
User Request
      ↓
GenAIOps Assistant
      ├── Infrastructure Agent
      ├── Prompt Management Agent
      ├── Evaluation Agent
      ├── Monitoring Agent
      ├── Trace Analyzer Agent
      ├── Troubleshooting Agent
      └── Documentation/Teaching Agent
```

MASTER ROADMAP

Phase	Focus	Main Outcome
Phase 1	Foundation Setup	Working local AI engineering environment
Phase 2	Infrastructure Automation	Azure deployment automation

Phase	Focus	Main Outcome
Phase 3	Prompt Management	Prompt versioning system
Phase 4	Agent Development	Deployable AI agent
Phase 5	Evaluation System	Automated AI quality checks
Phase 6	CI/CD Automation	GitHub Actions pipeline
Phase 7	Monitoring & Tracing	Production observability
Phase 8	Multi-Agent Architecture	Specialized operational agents
Phase 9	Enterprise Features	Governance + scalability
Phase 10	Autonomous GenAIOps Platform	Self-improving AI operations system

---

PHASE 1 — FOUNDATION SETUP

Goal

Prepare your development environment properly.

Skills You Will Learn

- Python environment management
- Azure authentication
- VS Code setup
- Git workflows
- Azure CLI
- Azure Developer CLI
- AI Foundry basics

Tasks

1. Install Core Tools

You should install:

- Python 3.11+
- VS Code
- Git
- Azure CLI
- Azure Developer CLI (`azd`)
- Docker Desktop (optional but recommended)

2. Configure GitHub

Create:

- GitHub account
- SSH key
- Git configuration

Practice:

```
git clone
git branch
git checkout
git commit
git push
```

3. Configure Azure

Practice:

```
az login
az account show
azd auth login
```

4. Create Project Structure

Suggested structure:

```
my-genaiops-agent/
├── agents/
├── prompts/
├── infra/
├── evaluations/
├── monitoring/
├── traces/
├── scripts/
├── docs/
└── tests/
```

Deliverable

A clean local development environment.

PHASE 2 — INFRASTRUCTURE AUTOMATION

Goal

Deploy Azure AI infrastructure automatically.

Skills You Will Learn

- Infrastructure as Code
- Bicep templates
- Azure provisioning
- Resource organization
- Environment isolation

Tasks

1. Create Bicep Templates

Deploy:

- Resource Group
- Azure AI Foundry
- Azure OpenAI
- Application Insights
- Storage Account

2. Use Azure Developer CLI

Practice:

```
azd init
azd up
azd env list
```

3. Environment Separation

Create:

- dev
- test
- prod

Deliverable

Fully automated Azure AI environment deployment.

PHASE 3 — PROMPT MANAGEMENT

Goal

Treat prompts like software assets.

Skills You Will Learn

- prompt versioning
- Git tagging
- prompt storage
- rollback workflows

Tasks

1. Create Prompt Files

Example:

```
prompts/
├── v1_system_prompt.txt
├── v2_system_prompt.txt
└── v3_system_prompt.txt
```

2. Build Prompt Loader

Python should dynamically load prompts.

3. Git Versioning

Practice:

```
git tag v1
git tag v2
```

4. Prompt Comparison Tool

Build small scripts that compare:

- token count
- structure
- changes

Deliverable

Version-controlled prompt management system.

PHASE 4 — AGENT DEVELOPMENT

Goal

Create your first operational AI agent.

Skills You Will Learn

- Azure AI Foundry SDK
- agent orchestration
- system prompts
- tool integration

Tasks

1. Create Base Agent

Capabilities:

- answer GenAIOps questions
- explain infrastructure
- troubleshoot errors

2. Add Tool Functions

Example tools:

- deployment checker
- Azure status checker
- configuration validator

3. Build Error Handling

Agent should:

- explain errors clearly
- suggest fixes
- identify root causes

Deliverable

Functional GenAIOps assistant agent.

PHASE 5 — EVALUATION SYSTEM

Goal

Automatically test AI quality.

Skills You Will Learn

- evaluation datasets
- automated evaluators
- regression testing
- benchmark scoring

Tasks

1. Build Evaluation Dataset

Examples:

- infrastructure questions
- troubleshooting prompts
- edge cases
- hallucination tests

2. Create Evaluation Pipeline

Evaluate:

- groundedness
- relevance
- latency
- safety

- hallucination risk

3. Store Evaluation Results

Save:

- scores
- timestamps
- version metadata

Deliverable

Automated evaluation engine.

PHASE 6 — CI/CD AUTOMATION

Goal

Automate deployments and evaluations.

Skills You Will Learn

- GitHub Actions
- AI deployment automation
- quality gates
- continuous integration

Tasks

1. Create GitHub Actions Workflow

Pipeline should:

```
Push Code
    ↓
Run Tests
    ↓
Run Evaluations
    ↓
Deploy If Passed
```

2. Add Quality Gates

Deployment blocked if:

- hallucination score too high
- latency too high
- evaluation score too low

Deliverable

Enterprise-style AI CI/CD pipeline.

PHASE 7 — MONITORING & TRACING

Goal

Observe and debug production AI systems.

Skills You Will Learn

- OpenTelemetry
- Application Insights
- tracing
- observability
- operational diagnostics

Tasks

1. Instrument Agent

Track:

- latency
- token usage
- failures
- prompt execution

2. Add Tracing

Trace:

- retrieval steps
- model calls
- tool execution

- prompt assembly

3. Create Monitoring Dashboard

Monitor:

- costs
- performance
- drift
- failures

Deliverable

Production-grade AI observability system.

PHASE 8 — MULTI-AGENT ARCHITECTURE

Goal

Split responsibilities across specialized agents.

Example Agents

Agent	Responsibility
Infra Agent	Azure provisioning
Prompt Agent	Prompt optimization
Evaluation Agent	AI testing
Monitoring Agent	Observability
Trace Agent	Root-cause analysis
Mentor Agent	Explain concepts

Deliverable

Collaborative multi-agent GenAIOps platform.

PHASE 9 — ENTERPRISE FEATURES

Goal

Add production governance and scalability.

Features

- RBAC
- secret management
- Key Vault
- audit logging
- deployment approvals
- policy enforcement
- budget monitoring
- multi-environment support

Deliverable

Enterprise-ready AI operations platform.

PHASE 10 — AUTONOMOUS GENAIOPS PLATFORM

Goal

Build self-improving operational AI systems.

Advanced Capabilities

- autonomous evaluations
- automatic rollback
- drift detection
- self-healing workflows
- intelligent optimization
- adaptive prompting

Deliverable

Autonomous AI operations assistant.

HOW WE WILL WORK TOGETHER

For every phase:

1. You implement step-by-step
2. You share:
3. code
4. screenshots
5. errors
6. logs
7. architecture
8. I help:
9. debug issues
10. explain concepts
11. improve architecture
12. optimize workflows
13. add enterprise patterns

IMPORTANT DEVELOPMENT PRINCIPLE

Do NOT try to build everything immediately.

We will build:

```
Simple
  ↓
Working
  ↓
Reliable
  ↓
Automated
  ↓
Scalable
```

This is how real enterprise AI systems evolve.

FIRST PRACTICAL IMPLEMENTATION TARGET

Your first working milestone should be:

```
A GenAIOps assistant that:
- explains Azure AI concepts
- loads versioned prompts
- runs locally
- deploys to Azure AI Foundry
- logs requests
- handles errors clearly
```

That will become the foundation for everything else.

RECOMMENDED TECH STACK

Area	Recommended Stack
Language	Python
Cloud	Azure
IaC	Bicep
CI/CD	GitHub Actions
Monitoring	Application Insights
Tracing	OpenTelemetry
Version Control	GitHub
Agent Runtime	Azure AI Foundry
SDK	Azure AI SDK
Vector Search	Azure AI Search
Secrets	Azure Key Vault

MOST IMPORTANT MINDSET

You are NOT building:

```
just a chatbot
```

You are building:

an operational AI engineering platform

That is the mindset of enterprise GenAIOps.